

Predict, Measure, Update: How Neural Networks Learn

Every neural network that has ever been trained — from a two-parameter line fit to a frontier language model — learns by repeating the same three steps. Make a prediction. Measure how wrong it was. Adjust the weights so that next time it will be a little less wrong. That is the entire lecture, and in a real sense the entire field of deep learning, compressed into a loop: *predict, measure, update*. Everything that follows is detail on those three verbs, and most of it is detail on the third.

This essay walks the loop in order. First, what goes in and what comes out. Second, the *loss function*, which turns “how wrong” into a single number. Third, *gradient descent*, the algorithm that uses that number to decide which way to move. Fourth, the two engineering ideas that make gradient descent feasible at scale: batching and backpropagation. Fifth, what the loop looks like in PyTorch. Finally, how the same loop trains a language model, and what changes when you stop training from scratch and start fine-tuning something that already exists.

I. Inputs and Outputs

A deep learning model is general in a way classical models are not. The input can be an image, a sound wave, or a passage of text; the output can be a category, a number, or an object in an entirely different modality — speech in, text out; English in, French out; a caption in, an image out. The model itself never sees pixels or words. It sees vectors of numbers.

Two translation steps bracket every model. *Encoding* turns a real-world input into a mathematical object: an image becomes a grid of brightness values; a sentence becomes a sequence of token IDs, then embeddings. *Decoding* turns the model’s output vector back into something a person can use: a vector of ten scores becomes “ankle boot”; a distribution over a vocabulary becomes the next word. The scales on either side need not match. An image may enter as pixel intensities between 0 and 255 and leave as a single 0/1 verdict — cow or not cow.

With the plumbing in place, the network makes its guess. Call that guess $\hat{y}$ (“y-hat”) and the correct answer y . Training is about the gap between them.

II. Measuring Performance: The Loss Function

A *loss function* collapses an entire set of predictions into one non-negative number. Near zero means the model is doing well; large means it is doing badly; negative never happens. In supervised learning y comes from an answer key, and the loss is summed over every example in the dataset.

Mean squared error

For regression — predicting a continuous quantity like a house price or a temperature — the workhorse is mean squared error:

$$\mathcal{L}_{\text{MSE}} = \sum_i (\hat{y}_i - y_i)^2.$$

Subtract, square, add up. An exact match contributes nothing; a large miss contributes a great deal.

Why square? Two reasons, one of necessity and one of taste. The necessity is that errors must be positive: overshooting by 7 and undershooting by 7 are equally wrong, and both should add to the loss rather than cancel. The taste is that squaring punishes large errors disproportionately — 70^2 is not ten times 7^2 but a hundred times. A cubic would keep the sign, so a negative error would *reduce* the loss; it is not a candidate.

The disproportion cuts both ways. Outliers get amplified and pull the whole model toward them, so in practice one filters extreme points first (beyond roughly 3.5 standard deviations, say) before training. The gentler alternative is mean absolute error, $\sum_i |\hat{y}_i - y_i|$, which treats a miss of 70 as only ten times as bad as a miss of 7. Hybrids exist — absolute below a threshold, squared above — but the choice between MSE and MAE is a low-priority one. If training is failing, the loss function is rarely the reason. The first thing to check is the learning rate, which we come to shortly.

Cross-entropy

For classification — predicting one of a fixed set of categories — the loss is cross-entropy:

$$\mathcal{L}_{\text{CE}} = - \sum_i p_i \log \hat{p}_i,$$

where p is the true distribution and $\hat{p}$ the predicted one. Its derivation lives in information theory and is out of scope here, but its behavior is the same as MSE's: a prediction close to the truth costs little, a prediction far from it costs a lot. Predict a 10% chance of sun on a day it rains and the loss is small; predict 95% sun and the loss is around 3.5. A naive alternative like $1 - \hat{p}$ would point in the right direction, but cross-entropy is preferred because it falls out of the mathematics rather than being invented.

The reason cross-entropy matters for this course is that *it is the loss language models are trained on*. Predicting the next token is classification — just classification over a vocabulary of fifty thousand categories instead of ten.

III. The Update Step: Gradient Descent

Now we can say precisely what training means. The loss depends on the weights; training is the search for the weights that make the loss as small as possible. “Line of best fit” is exactly this phrase — *best* means minimum loss — applied to two parameters instead of billions.

The algorithms that perform this search are called *optimizers*, and every modern one is a variant of a single idea: gradient descent. Momentum, RMSProp, Adam, AdamW — each is gradient descent with a few extra sprinkles, and each can be understood in half an hour once the base idea is clear.

The gradient

The derivative of a function is its slope at a point. On a parabola the slope depends on where you stand: steep on the sides, zero at the bottom. The *gradient* is simply the derivative taken in every dimension at once — a vector whose j th entry is the slope with respect to the j th weight:

$$\nabla_{\theta}\mathcal{L} = \left[\frac{\partial\mathcal{L}}{\partial\theta_1}, \frac{\partial\mathcal{L}}{\partial\theta_2}, \dots, \frac{\partial\mathcal{L}}{\partial\theta_n} \right].$$

The core principle is that *the sign of the slope tells you the direction of increase*. Positive slope: moving right raises the loss. Negative slope: moving right lowers it. So to decrease the loss, move against the slope. If you are left of the minimum the slope is negative and you step right; if you are right of it the slope is positive and you step left. Either way you approach the bottom. Because this holds in every dimension independently, the same rule works for one weight or for a hundred billion. Written out, one step of gradient descent is

$$\theta \leftarrow \theta - \eta \nabla_{\theta}\mathcal{L}(\theta),$$

where η is the learning rate, discussed below.

The loss landscape

It helps to be clear about which graph we are drawing. In the first lecture the picture was data on the horizontal axis and predictions on the vertical: a line through a scatter of points. This lecture's picture is different. The axes are the *weights*, and the height is the *loss*. For $y = ax + b$ there are two weights, so the loss is a surface over a plane — a three-dimensional landscape — and training is a ball rolling down that landscape toward its lowest valley. A model with a single weight, $y = ax$, gives a two-dimensional curve.

The ball metaphor also exposes the classic worry. Gradient descent stops wherever the slope is zero, and a slope of zero marks any minimum, local or global. A ball can settle in a shallow dip well above the true valley floor. In deep learning this matters less than intuition suggests: the two ideas in the next section — momentum and the randomness of small batches — both help the ball roll out of shallow dips, and in very high-dimensional landscapes true traps are rarer than they look on a 3D plot.

The learning rate

The learning rate η is a scalar — a single number, not a vector — that scales how far you move at each step. It is the most consequential knob in training and the hardest to set by reasoning alone.

Too small, and the steps are microscopic; the loss creeps downward so slowly that convergence takes, in the lecturer's phrase, a million years. Too large, and each step overshoots the minimum, landing on the far slope, from which the next step overshoots back — and if the overshoot grows, the loss climbs instead of falling. In between lies a regime where steps overshoot but shrink, and the model converges anyway.

The loss curve diagnoses which regime you are in. Loss flat at the same value for hundreds of iterations: the learning rate is too small. Loss skyrocketing: too large. What you want to see is a

loss that is, on the whole, going down. The optimizer variants add refinements on top — *learning rate decay* shrinks η as training proceeds so that late steps are fine-grained, and *momentum* carries velocity from previous steps. AdamW manages its own per-parameter scaling.

The foggy mountain

All of this fits into a single image. You are on a mountain in dense fog, and you want to reach the valley. You cannot see the landscape; you can only feel the ground under your feet. Worse, measuring the steepness requires a slow, expensive instrument, so you can only afford to take a reading every so often. Your strategy: take a reading, walk downhill for a while, take another reading, repeat.

The mapping is exact. You are the algorithm. The path you walk is the sequence of weights explored. The steepness you measure is the gradient. The instrument is differentiation — the procedure that computes the gradient. And the distance between readings is the step size, which equals the learning rate times the gradient. The fog is why nobody ever sees the whole landscape: in a space of millions of dimensions there is no map to consult, only the slope where you stand. Asking whether you could reuse the map if you already knew where the minimum was is like asking whether you can find the largest element of a 50,000-entry array without looking through it. You cannot.

IV. Making It Feasible: Batches and Backpropagation

Two practical questions remain. How much data should each gradient reading use? And how is the gradient actually computed?

Batch size

Every update requires a gradient, and the gradient depends on the data. The choice is how much of the data to look at before moving.¹

Batch gradient descent looks at everything: it averages the gradient over the entire dataset and takes one step. This is the true gradient, and the path it traces is clean and direct. *Stochastic gradient descent* (SGD) looks at one example: it computes the gradient from a single data point and steps immediately. The path is noisy and zigzags, because each individual gradient is a rough estimate of the true one. *Mini-batch* gradient descent sits anywhere in between — 32 examples, say — and is what almost everyone actually runs. Batch sizes are usually powers of two, for reasons of GPU memory layout.

The trade-off is efficiency against accuracy. The stochastic path looks messier, but the bottleneck in training is computing the gradient, and a gradient over one example is cheap. Fifty one-second steps that each point roughly the right way beat three ten-minute steps that each point exactly the right way. The noise also has a second virtue: it jostles the ball, which is one of the reasons local minima are less of a problem in practice than in theory. The lecturer’s preference is for small batches — 32, 64, or 128 — with a caveat worth stating: in most research and production settings,

¹A note on vocabulary. In lecture, “epoch” was used to mean one pass through the loop, i.e. one update. Standard usage reserves *epoch* for a full pass through the dataset and calls each update a *step* or *iteration*; the notebook’s outer loop uses the standard meaning.

batches in the hundreds to thousands are standard, and a batch size of 1 is usually a workaround for limited memory rather than a principled choice.

That memory constraint is the practical driver. The current batch must fit on the GPU. A modest card forces small batches; a large one permits large ones. In pseudocode, the stochastic loop is: initialize the parameters; then, repeatedly, draw one random example (with replacement), compute the loss between prediction and truth, compute the gradient ∇ , and subtract it from the parameters.

Backpropagation

In practice, computing the gradient is one line: `loss.backward()`. The name is a clue. The algorithm behind it is *backpropagation*, and it is worth understanding what the line does even if you never implement it.

What we want is $\partial\mathcal{L}/\partial\theta$ for every weight θ : how much the loss moves if that weight is nudged. A neural network is a nested composition of functions — a weighted sum, wrapped in an activation, wrapped in another weighted sum, wrapped in another activation, layer after layer. The derivative of a composition is given by the chain rule, applied repeatedly:

$$\frac{\partial\mathcal{L}}{\partial\theta^{(1)}} = \frac{\partial\mathcal{L}}{\partial h^{(L)}} \cdot \frac{\partial h^{(L)}}{\partial h^{(L-1)}} \cdots \frac{\partial h^{(2)}}{\partial h^{(1)}} \cdot \frac{\partial h^{(1)}}{\partial\theta^{(1)}}.$$

Read from right to left, each factor is the derivative of one layer with respect to the layer before it. The essential observation is that each layer’s contribution depends on the layers *after* it, and the only layer whose derivative is known at the outset is the last one — the loss itself. So you must work backwards, from the output toward the input. Hence the name.

Why is it efficient? Because the calculation is recursive and the intermediate results are shared: the derivative through layer 5 is needed for computing the derivative of layers 4, 3, 2, and 1. Backpropagation is dynamic programming — compute each intermediate once, store it, and look it up in constant time rather than recomputing it. This works because a neural network is a directed acyclic graph, so there is a well-defined order in which to fill the table. The calculus prerequisite is small; the first few videos of any introductory series, together with the third and fourth of 3Blue1Brown’s neural network series, cover everything needed.

One honest caveat from the discussion afterward: none of this comes with guarantees. That randomness helps, that the ball escapes local minima, that the loss will fall — these are empirical regularities, not theorems. The discipline is to watch the loss and the validation accuracy. If accuracy sits at the chance baseline (25% on four classes), something is broken, and the learning rate is the first suspect.

V. The Loop in Code

Everything above compresses into a few lines of PyTorch. The setup is standard: a hosted GPU on Colab (a free T4 suffices), the `torch` and `torch.nn` imports, and a dataset — here FashionMNIST, ten classes of clothing images, the sibling of the handwritten-digit MNIST. A `DataLoader` wraps the dataset and handles batching, shuffling, and parallel loading; it is a code convenience, not a concept. The first sanity check is shape: a batch comes out as `[64, 1, 28, 28]` — batch size, channels, height, width — with 64 labels to match. If the dimensions add up, things are probably right; dimension mismatches are the first thing to look for when they are not.

The model is a stack: `Linear(784→512)`, `ReLU`, `Linear(512→512)`, `ReLU`, `Linear(512→10)`, where $784 = 28 \times 28$ is the flattened image. Its `forward` method flattens the input, applies the stack, and returns *logits*, the raw unnormalized scores. In real projects you rarely write this from scratch; you import a pretrained architecture.

The training loop reads best as a sequence of instructions to objects:

```

loss_fn = nn.CrossEntropyLoss()           # classification -> cross-entropy
optimizer = SGD(model.parameters(), lr=1e-3) # optimizer: weights + learning rate

model.train()                             # model, switch to learning mode
for batch, (X, y) in enumerate(dataloader):
    X, y = X.to(device), y.to(device)      # data, move onto the GPU
    pred = model(X)                        # model, make your prediction
    loss = loss_fn(pred, y)                # loss function, score vs. truth
    loss.backward()                        # loss, compute the gradient (backprop)
    optimizer.step()                       # optimizer, move the weights one step
    optimizer.zero_grad()                  # optimizer, clear the gradient

```

The function takes four things — data, model, loss function, optimizer — which are exactly the four boxes in the training diagram, and the body is the three conceptual steps: predict, compute the loss, compute the gradient and step. The one non-obvious line is `zero_grad`. PyTorch accumulates gradients by default, which is useful in large runs; for ours, we clear them every iteration.

Validation is the same loop with the learning turned off. Calling `model.eval()` switches the model into evaluation mode; wrapping the loop in `torch.no_grad()` tells PyTorch not to track gradients; and accuracy is the fraction of examples whose highest-scoring class matches the label: `pred.argmax(1) == y`, cast to ones and zeros, summed, and divided by the dataset size. (The notebook calls this the test loop; it is really validation.) Five epochs on FashionMNIST moved validation accuracy from 48% to 53% with validation loss falling — unimpressive numbers, but the curve is heading the right way, and printing both training and validation loss is the habit to build.

Two housekeeping points. Save weights with `torch.save(model.state_dict(), path)` and reload them with `model.load_state_dict(torch.load(path))`. And save to permanent storage — a mounted Google Drive — because Colab wipes its local directory when the runtime dies.

VI. Training a Language Model

Everything so far assumed an answer key: a label y for each example. Language models are trained on the raw internet, where no one has labeled anything. How does the loop run without y ?

The answer is *self-supervised learning*, which sits between supervised (labels) and unsupervised (no labels): the model labels the data itself, by predicting one part of the input from another. Take a sentence, hide a word, and ask the model to predict it from everything that came before. A sentence of n tokens yields $n - 1$ training examples, each with a built-in answer. The pre-training pipeline is therefore: take the internet, apply basic filtering (toxic content and the like), and generate next-token prediction problems by the trillion. No human labelers are required, though frontier labs invest heavily in data triage and in upsampling the most informative sources.

But what, exactly, is the ground truth? Consider “the plate was a beautiful shade of ____.” The true next-token distribution — 30% gray, 50% blue, a scattering of others — is unknowable; no one wrote it down. The surrogate is to treat the token that actually appeared as 100% correct:

a *one-hot* vector with a 1 at the observed token and 0 everywhere else. Cross-entropy against a one-hot target collapses to a single term,

$$\mathcal{L}_t = -\log \hat{p}(x_t^*),$$

the negative log-probability the model assigned to the token that came next. Each individual target is wrong — the true distribution surely put some mass on gray — but over internet scale the errors average out, exactly as Monte Carlo sampling estimates a distribution from draws. The cost of the surrogate is that it rewards confidence: the model is pushed toward certainty on every example, and this is one contributor to the *miscalibration* of trained language models.

With this, the picture from lecture 2 is complete. That lecture covered post-training — SFT, RLHF, RLVR — the layer that turns a raw predictor into an assistant. This lecture covered pre-training and the mechanics of training itself. The only component still treated as a black box is attention.

VII. Fine-Tuning, and the Art of Practice

You will almost never train from scratch. Frontier pre-training runs cost billions of dollars; no student project competes. The one exception is small *model organisms* built for interpretability research — with the caveat that what holds for a tiny model does not necessarily hold at scale.

What you will do is *fine-tune*: take an existing model’s final weights and train for a few more epochs on your own specialized data. This is not a different kind of training. It is the same loop, with a different starting point: instead of initializing the weights at random, you initialize them at values that already understand language, or images. This is also called transfer learning, and it is why 5,000 math problems can teach a model basic math — but only a model that already reads English. A brilliant exchange student who cannot read the word problems will not learn much from them.

The case for fine-tuning is clearest in the messy middle. A hospital has 10,000 clinical notes dense with system-specific acronyms. A general pretrained model underperforms on them; a from-scratch model either memorizes (if large) or is too weak (if small). Fine-tuning threads the needle. The alternative, worth knowing, is retrieval-augmented generation — leave the model alone and hand it the relevant notes at query time.

Fine-tuning carries two risks. *Overfitting* is the familiar one: train too long and the model memorizes your data rather than learning from it. *Catastrophic forgetting* is subtler and unsolved: as the model learns the new thing, it gets worse at the old things. The research field that attacks it is called continual learning, and its lack of a clean solution is a standard argument in the skeptic’s case against current AI.

Practical guidelines

The lecture closed with advice that is hard-won rather than derived. *The learning rate is the biggest lever*. It is also nearly impossible to reason out from first principles — it could be anywhere from 10^{-3} to 10^{-15} — so ask mentors and PIs, who carry intuition from many runs. When fine-tuning, a good starting point is the *final*, decayed learning rate that the pretrained model ended on. Use AdamW. Start the batch size at 1 and grow it. Deliberately overfit first: underfitting means the pipeline learned nothing and you cannot tell what is wrong, while overfitting proves the pipeline works and you can then pull back. Monitor validation loss with Weights & Biases rather than

raw print statements. Save weights every epoch, to permanent storage, because runtimes die. For inference memory, a rule of thumb is twice the parameter count in gigabytes — a 7B model wants roughly 14–15 GB in half precision, far less when quantized to 4 bits.

Hyperparameters

A *hyperparameter* is a setting fixed before training that training itself cannot learn: the learning rate, which governs how training proceeds, or the number of layers and neurons per layer, which decide what is being trained at all. They sit one meta-level above the loop.

Tuning them by hand is feasible only for a handful of settings on small models. *Grid search* tries every combination, and the arithmetic is unkind: ten learning rates, three depths, and three widths give 90 runs, which at five hours each is 450 hours — and the space grows exponentially with each added dimension. *Random search* samples a subset (ten of the 90, say) and, naive as it sounds, is the practical default. *Bayesian optimization* is smarter and is used at places like Meta’s research labs, but is overkill for most projects; so, often, is the tuning framework Optuna.

One Loop, Billions of Weights

Step back and the lecture collapses to its opening sentence. Predict: encode the input, run it through the network, decode a guess. Measure: compare the guess to the truth with a loss — squared error for numbers, cross-entropy for categories, cross-entropy against a one-hot next token for language. Update: compute the gradient by backpropagation, scale it by the learning rate, step against it, and repeat — on one example at a time or a thousand, from random weights or from a model someone else already trained.

That is the whole machine. The four lectures of the foundations block — machine learning basics, post-training, neural networks, and now training — are the complete picture, with attention as the single remaining black box. The advice from here is not to keep learning horizontally. The foundation is laid. Go straight into research.